

UT5. Programación orientada a objetos en PHP

DESARROLLO WEB EN ENTORNO SERVIDOR (DWS)

Contenidos

1. Orientación a objetos en PHP.

- Características de orientación a objetos en PHP.
- Creación de clases.
- Utilización de objetos.
- Mecanismos de mantenimiento del estado.
- Herencia.
- Interfaces.
- Ejemplo de POO en PHP.

2. Programación en capas.

- Namespaces en PHP.
- Gestionar dependencias.
- Separación de la lógica de negocio.
- Generación del interface de usuario.

Programación orientada a objetos

UT04. DESARROLLO DE APLICACIONES WEB CON PHP

Programación orientada a objetos.

Creación de clases.

Para declarar una clase se utiliza la palabra reservada **class**. Los atributos se declaran utilizando su nombre, los modificadores que pueda tener y opcionalmente un valor por defecto:

```
class MiClase {  
    private $att1 = 10;           // con valor por defecto  
    private $att2;               // sin valor por defecto  
    private static $att3 = 0;    // estático  
}
```

Los atributos y métodos se pueden declarar como estáticos, de manera que no habrá uno por objeto, sino por clase (static).

Programación orientada a objetos.

Creación de clases.

Se puede crear un constructor **__construct()** éste método es un “método mágico” (nombre reservado a tareas concretas y que comienzan por dos guiones bajos “__”).

Para crear un nuevo objeto se utiliza el operador **new** seguido del nombre de la clase:

```
$obj = new MiClase();
```

Para las propiedades y métodos se pueden utilizar los siguientes modificadores de visibilidad (por defecto serán públicas):

- **public**: Se pueden utilizar desde dentro y fuera de la clase.
- **private**: Pueden emplearse desde la propia clase.
- **protected**: Se pueden utilizar dentro de la propia clase, las derivadas y las antecesoras.

Normalmente las propiedades se declaran como privados y se crean métodos públicos para acceder a ellos.

```
$objeto ->propiedad;           // propiedades
```

```
$objeto ->método(argumentos);  // métodos
```



Programación orientada a objetos.

Mantenimiento de estados.

Para almacenar objetos en las sesiones abiertas por un usuario, éstos se deben serializar.

La función **serialize()** devuelve un string que contiene una representación lineal de cualquier valor que puede ser almacenado en PHP.

La función **unserialize()** puede utilizar ese string para recrear el valor original de la variable a partir de su representación lineal.

Si una aplicación serializa objetos se recomienda que la aplicación incluya las definiciones de las clases de los objetos serializados en cada página. No hacerlo podría resultar en un objeto deserializado sin su definición de clase (clase tipo **__PHP_Incomplete_Class_Name**).

Veamos un ejemplo **serialize.php**.

Fuente: <https://www.php.net/manual/es/language.oop5.serialization.php>



Programación orientada a objetos. Herencia.

La herencia es uno de los principios de la POO y PHP la implementa en su modelo de objetos.

Cuando una clase es extendida, la clase hija hereda todos los métodos públicos y protegidos, propiedades y constantes de la clase padre.

Los métodos son conservados mientras una clase hija no los sobrescriba.

Los métodos privados de una clase padre no son accesibles para la clase hija. Por ello, las clases hijas deben implementar un método privado por sí mismas.

La visibilidad de los métodos, propiedades y constantes puede ser relajada.



Programación orientada a objetos.

Interfaces.

Una interface de objetos permite crear código que especifica qué métodos y propiedades una clase debe implementar sin tener que definir la implementación de éstos métodos y propiedades.

Las interfaces comparten el mismo espacio de nombres que las clases, traits y enumeraciones, de modo que una vez definidas, su nombre no puede ser usado.

Los métodos declarados en una interfaz deben ser públicos.

Para definir una interface se utiliza la palabra reservada **interface**.

Para implementar una interfaz, se utiliza el operador **implements**.

Las interfaces también se pueden extender como las clases, utilizando el operador **extends**.

Veremos un ejemplo en **interfaz.php**



Programación en capas

UT04. DESARROLLO DE APLICACIONES WEB CON PHP

Programación en capas

La programación en capas ayuda a separar las distintas partes de una aplicación.

Existen varios métodos que permiten separar la presentación de la lógica de negocio. El más extendido es el MVC (Modelo Vista Controlador). Este patrón pretende dividir el código en tres partes:

- **Modelo:** Encargado de manejar los datos propios de la aplicación.
- **Vista:** Encargada de la interacción con el usuario (es la correspondiente a la interfaz con el usuario).
- **Controlador:** Encargada de decidir que se realizará a razón de las acciones que el usuario selecciona en la vista.

Programación en capas.

Namespaces

Namespaces: Representan un medio para encapsular elementos, es decir, es una forma de organizar y evitar conflictos entre nombres de clases, interfaces, funciones o constantes que pueden tener el mismo nombre pero pertenecer a contextos diferentes.

Por ejemplo: Una estructura de directorios hace que un archivo esté localizado en una carpeta o subcarpeta determinada, no pudiendo existir dos archivos con el mismo nombre en la misma.

Namespace: es como una “carpeta virtual” dentro del código PHP que permite agrupar clases relacionadas bajo un mismo nombre lógico.

Para definirla se usa la palabra clave **namespace** por ejemplo:

```
namespace App;
```

Veremos un ejemplo con el código **namespace_Persona.php**

Referencia: <https://www.php.net/manual/es/language.namespaces.php>



Programación en capas.

Composer

Composer es un gestor de dependencias en proyectos usado en la programación de aplicaciones PHP. Es el equivalente a maven en java.

Se utiliza para:

- Instalar y actualizar librerías de terceros.
- Autocargar tus propias clases.
- Organizar dependencias y versiones de forma automática.

Sus principales ventajas:

- Es simple de utilizar
- Cuenta con un repositorio propio.
- Disminuye significativamente problemas de cambio de ejecución.
- Actualiza las librerías a nuevas versiones de manera fácil.



Programación en capas.

Separación de la lógica de negocio

Blade (template engines): es un sistema de plantillas que permite la generación de HTML dinámico con una sintaxis limpia.

Sus características son:

- Herencia de vistas (@extends, @section, @yield).
- Directivas personalizadas (@if, @foreach,...)
- Cachea las plantillas para un mejor rendimiento.
- Integrado totalmente con Laravel.

Fuente: <https://laravel.com/docs/blade>